Review a pull request

Week 7 Friday activity · medium · about 15 minutes a pull request

Brian C. Keegan, Ph.D. · Department of Information Science, University of Colorado Boulder

October 2, 2026

What a review is

A review reads a classmate's proposed change to the book. It says whether the change should go in, and what must change first. You read and you comment. You do not run code, and you do not change the pull request.

Your chapter's board in week 7's slides lists the pull requests to review. A line like `#88 and #93` means that two pull requests make the same change. Read them together, and say which one to keep.

Work in pairs. One partner works in the browser, and the other reads this handout aloud.

The three Friday activities

Triage an issue	easy
Review a pull request	**medium**
Issue to pull request	hard

Do not

- click **Merge pull request**, even if GitHub shows it;
- change someone else's pull request: comment on it;
- close the pull request.

The instructor merges, closes, and resolves conflicts.

1 · Read what it proposes

1. Sign in to GitHub.
2. Open the pull request. Its address is `https://github.com/cuinfoscience/Web-Data-Science-Book/pull/` followed by its number, for example `.../pull/104`.
3. On the **Conversation** tab, read the description. The template asks for **Closes #N** (the issue it fixes), **Location**, **Problem**, **Why**, **Change**, **AI assistance**, and **Checks**.
4. If it names an issue, open the issue. Does the pull request do what the issue asks for?
5. Read the comments. If another pair has reviewed it, read their review. Add only what is new.

2 · Look at the checks

Scroll to the bottom of the **Conversation** tab.

You see	What it means
Render with a green ✓	The book still builds with this change.
Render with a red ×	The book does not build. This is a must-fix: say so in your review.
Notebook sync with a red ×	Normal for a change made in the browser. Ignore it. The instructor regenerates the notebooks after merging.
Trope lint	It never fails. Ignore it.
This branch has conflicts that must be resolved	Do not try to fix it. The instructor resolves conflicts. You can still review the change.

3 · Read the change

Click **Files changed**. Removed lines are red (1), and added lines are green (2) (Figure 1). Check five things:

1. **The right file.** Only the chapter's `.qmd` file changes. A file in `notebooks/` or `images/` must not change.
2. **One change.** It changes what the description says, and nothing else.
3. **Correct.** Open the same section of the book in a second tab, at `https://cuinfoscience.github.io/Web-Data-Science-Book/`. Are the facts right? Does each new link open the right page? Does new code use the chapter's names, such as `HEADERS`?
4. **Clear.** Can a classmate who missed class understand it?
5. **The Markdown.** The checks do not see these mistakes, so read for them:

- a link is `[text](https://...)`, with no space between `]` and `(` (3);
- a heading (`##`) or a callout (`:::`) needs an empty line above it (4). A heading straight after a callout's `:::` line is the callout's title, and needs none;
- a code block opens and closes with three backticks.

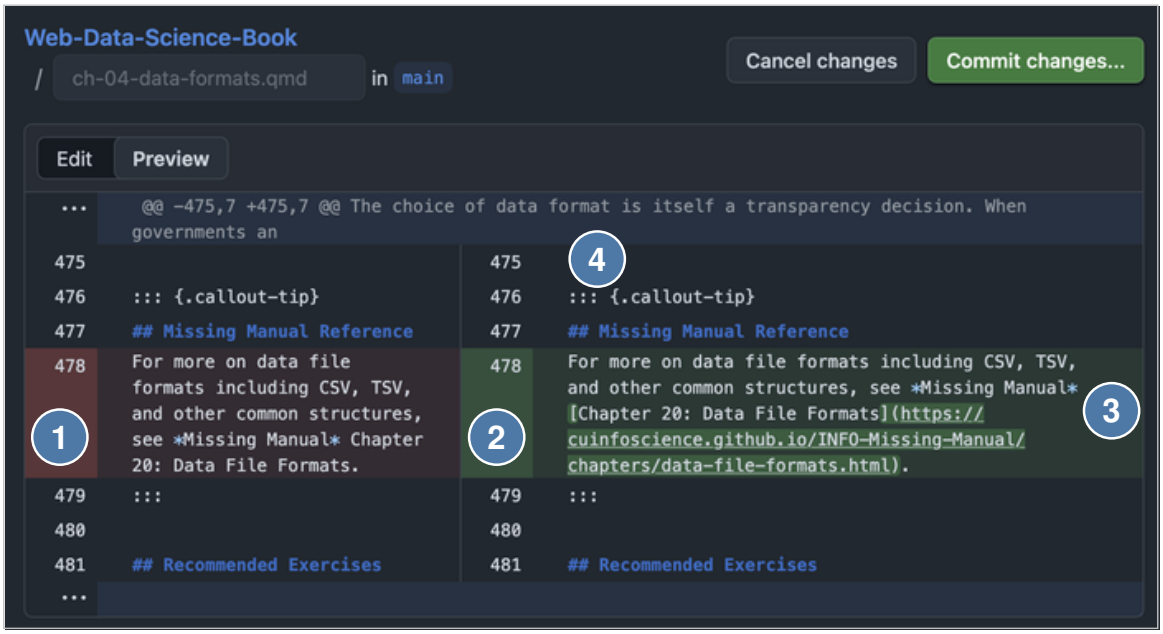


Figure 1. One change, line by line (here week 4's example, in the web editor's Preview; **Files changed** uses the same colors). The removed line (1) is red, and the added line (2) is green. The new line writes a link as `[text](https://...)` (3). The callout keeps the empty line above its `:::` line (4).

4 · Comment on a line

1. Move the pointer over a line. A blue **+** shows beside it. Click it.
2. Write one point: what is wrong, and what to change. To propose exact words, click **Add a suggestion** (the ± button), and edit the line in the box.
3. Click **Start a review**. For your next comments, click **Add review comment**. Nobody sees them until you submit the review.

5 · Submit the review

1. Click **Review changes**, at the top right of **Files changed**.
2. Write a summary in the shape on the right.
3. Select the option that matches your verdict:
   - **Approve** for *merge*;
   - **Request changes** for *needs changes*;
   - **Comment** for *close*, or if you are not sure.
4. Click **Submit review**.

Your summary

```
Verdict: merge / needs changes / close
Must fix: at most two points
Nice to have: optional
Reviewed with @partner
```

Give the verdict *close* for a duplicate (another pull request makes the same change) or for a change that the book does not need. Say why, and thank the author.

A good review is kind and specific. It names the line, it keeps must-fix points apart from nice-to-have points, and it ends with a verdict.

Next. Review the next pull request on your board. A pull request with a review already can use a second one.